A Comparative Study of TCP Load-Balancing and Server Concurrency Architectures

Computer Networks Project
Final implementation and benchmark snapshot: October 24, 2026

***Abstract*—This study compares a from-scratch Layer-4 TCP load balancer with two research works that overlap with its central concerns: Rawls and Salehi's evaluation of HAProxy load-balancing methods, and Pariag *et al.*'s comparison of event-driven, threaded, and hybrid web-server architectures. The comparison is organized around shared parameters rather than treating the papers as direct replicas of the project. The first work shares the concern of how load-balancing policy behaves under different workloads and backend configurations; the second shares the concern of how connection-processing architecture affects performance. The project itself implements Round Robin, Least Connections, and consistent hashing in both a blocking thread-per-connection proxy and a nonblocking `epoll` proxy. Its final short-run benchmark reports throughput, latency, CPU-set utilization, context switches, idle connection capacity, failover behavior, routing properties, and TCP half-close correctness. At 800 connections, four-worker epoll achieved 253,129 requests/s, compared with 165,938 requests/s for a four-core threaded proxy. Epoll also held 30,000 idle sessions with three process threads. However, the throughput build was unoptimized and each cell had one three-second sample. The literature studies use different systems and workloads, so their numerical results are not directly comparable with these measurements. The contribution of this paper is a structured comparison of common research dimensions, an evidence-based interpretation of the project results, and a precise account of what remains untested.**

***Index Terms*—TCP load balancing, HAProxy, server architecture, epoll, throughput, scalability, benchmarking**

I. Introduction

Load-balancing performance is shaped by multiple interacting factors: backend-selection policy, connection-processing architecture, offered workload, backend behavior, and resource constraints. A fair comparison must therefore distinguish questions that overlap from questions that do not. This paper compares the final TCP load-balancer project with two selected studies: Rawls and Salehi's technical report on HAProxy load-balancing methods [1], and Pariag *et al.*'s peer-reviewed comparison of web-server architectures [2].

The project and the first study overlap in the role of routing policy and workload/backend conditions. The project and the second overlap in the role of concurrency architecture. They do not use the same software, workload, or experimental protocol, so reported numbers should not be ranked against each other as if they were obtained in a controlled head-to-head experiment. The objective is to compare methods, shared parameters, findings, and limitations.

II. Systems Under Comparison

A. Project implementation

The project is a C++17 Layer-4 TCP proxy. It accepts a client TCP connection, selects a backend, opens an upstream connection, and relays bytes in both directions. Its two data-plane implementations share backend-health and routing components:

- **ProxyCore:** blocking thread-per-connection; approximately two relay threads per proxied session.
- **EpollCore:** nonblocking sockets and a level-triggered `epoll` loop, with optional multiple `SO_REUSEPORT` workers.

The routing choices are Round Robin, Least Connections, and consistent hashing of client IPv4 addresses over a ring with 100 virtual nodes per backend. Active checks test TCP listener availability, and passive retries apply only to initial upstream connection failures. Established streams are not replayed after an upstream failure.

B. Selected research studies

Rawls and Salehi examine HAProxy balancing methods under differing request loads, GET/POST workloads, and homogeneous versus heterogeneous backends. Their technical report uses JMeter with a WordPress three-tier application and emphasizes that the best method depends on workload and backend characteristics [1]. It is the closest selected source for the project's routing-policy and backend-behavior dimensions, but it is not a direct comparison of the project's implementation.

Pariag *et al.* compare an event-driven μserver, a thread-per-connection Knot server using Capriccio, and a hybrid C++ WatPipe server. Their study uses `httperf` with partially open workloads and dynamic and static content. It is not a load-balancer study, but it is relevant to the project's comparison of blocking threaded and event-driven connection handling [2].

III. Comparison Framework

Table I identifies the dimensions that can be compared without implying that the systems or workloads are identical.

The comparison matrix is deliberately conservative. The project reports detailed raw measurements, but they come from a short local test. The two studies provide research context for interpreting dimensions and experimental design; their published results cannot be used as numerical baselines for this project's `wrk` values.

TABLE I
SHARED COMPARISON DIMENSIONS AND EVIDENCE BOUNDARIES

Dimension	This project	Rawls and Salehi [1]	Pariag *et al.* [2]
Primary question	Compare TCP proxy architectures and test routing/failover behavior	How HAProxy balancing methods respond to workload and backend differences	How event-driven, threaded, and hybrid server architectures perform
System type	Layer-4 TCP load balancer	HAProxy load balancer in a WordPress three-tier setup	Web servers, not a load balancer
Architecture dimension	Thread-per-connection vs. `epoll`; single and four workers	Not the principal comparison in the selected report	Central comparison: event-driven, threaded, hybrid
Routing / scheduling dimension	Round Robin, Least Connections, consistent hashing	Multiple HAProxy balancing methods	Request scheduling is not the same load-balancing-policy comparison
Workload	`wrk`; local small HTTP response; concurrency 50/200/800	JMeter; GET/POST, varied request loads and backend heterogeneity	`httperf`; partially open workloads; static/dynamic content
Measured outcomes	RPS, latency percentiles, CPU-set busy, context switches, idle sessions, failover	Workload-dependent behavior of balancing methods	Server throughput and response behavior under tested workloads
Direct numeric comparability	Baseline for this project's internal modes only	No: different software, application stack, harness, and workload	No: different system purpose, implementation, and workload

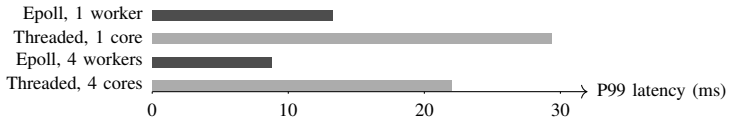


Fig. 1. P99 latency at 800 concurrent connections in the project's own test.

TABLE II
SELECTED RESULTS AT 800 CONNECTIONS

Mode	RPS	P99 (ms)	CPU busy (%)
Control	130,427	8.17	62.3
Epoll, 1 worker	82,423	13.28	98.4
Threaded, 1 core	56,715	29.39	96.2
Epoll, 4 workers	253,129	8.81	95.5
Threaded, 4 cores	165,938	22.03	91.7

IV. PROJECT EXPERIMENTAL METHOD

The final benchmark ran on a 12-logical-CPU Linux host with 15 GiB RAM. The host-mode run used separate CPU sets for proxy (0 or 0–3), load generator (4–7), and dummy backends (8–11). The compiler flags were `-g -std=gnu++17 -Wall -Wextra -Werror -Wpedantic`; no optimization flag was used. Docker was unavailable.

The active-load tests used `wrk 4.2.0`, four generator threads, a one-second warmup, and a three-second measurement at 50, 200, and 800 connections. Each cell had one repetition. The backend returned a small local HTTP response. Idle tests held connections for five seconds and sampled process RSS/PSS and descriptor/thread counts. Failover tests sent 20 probes/s and killed one backend for 2.5 seconds across 20 trials per mode. Routing tests used five runs per policy. A separate test verified a 1 MiB request/response exchange across TCP half-close.

V. RESULTS AND COMPARISON

A. Architecture performance

At 800 connections, one-worker epoll achieved 82,423 requests/s, compared with 56,715 for the single-core threaded proxy. Four-worker epoll achieved 253,129 requests/s, compared with 165,938 for the four-core threaded proxy. These are approximately 45.3% and 52.5% higher observed throughput, respectively, in this run. The four-worker comparison also showed P99 latency of 8.81 ms for epoll and 22.03 ms for the threaded proxy. Figure 1 summarizes the P99 comparison.

These results are consistent with the architectural question explored by Pariag *et al.*: concurrency model can affect throughput and response behavior. However, the project does not reproduce their server implementations or workload, so their reported performance differences cannot validate or contradict these exact values. The project's one-sample, unoptimized run is a preliminary observation rather than a statistically established performance ranking.

B. Routing-policy behavior

The project tests share a narrower point of contact with Rawls and Salehi's study. Both consider backend selection, but the project verifies selected functional properties rather than comparing policies across application workloads. Round Robin produced a 10/10/10 split over 30 connections in all five trials. Least Connections distributed eight new held sessions as 3/3/2 when two sessions were retained on backend 9001. Consistent hashing preserved the backend assignment of each of 30 client IPs across reconnects; the observed distribution was 12/9/9. These checks establish the specified behaviors in the test scenarios, not universal policy superiority.

Rawls and Salehi's varied GET/POST workloads and homogeneous/heterogeneous backends highlight an important gap: the current project does not measure how routing methods respond to different request costs or backend capacities. Its small dummy response and short local tests cannot establish which policy would be best for a real application. A future experiment could reproduce the same workload across all three project algorithms, vary backend service time and capacity,

and report per-backend request share, tail latency, and failure impact.

C. Scalability and failure handling

Epoll established 10,000, 20,000, and 30,000 idle sessions with three process threads. At 30,000 connections it held 60,005 file descriptors and showed a 7.10 MB increase in process PSS over baseline. The threaded implementation passed the 5,000 target with 10,003 threads and 121.03 MB PSS growth, but failed its 10,000 target: it established 9,213 sessions and recorded an incomplete active-probe result. Its 150.90 GB `VmSize` is virtual address space, not physical RAM. Neither PSS nor RSS includes kernel TCP socket memory.

In 20 active-check-only failover trials, the mean dropped-probe count was 3.65 ± 2.16, with a range of 0–7. With passive retry enabled, all 20 trials recorded zero dropped probes under the test schedule. Both architectures passed the 1 MiB TCP half-close test with exact response payload/hash matching. These are project-specific tests; the selected literature studies do not provide directly matched idle-connection or failover measurements for comparison.

VI. Discussion

The common thread across the three works is that performance is conditional on the tested system and workload. Rawls and Salehi motivate examining backend-selection methods under more than one request mix or backend profile. Pariag *et al.* motivate examining the cost of connection-processing architecture. The project contributes an implementation where these questions can be explored together, but its present measurements address them at different levels: architecture is compared with throughput and latency; routing is checked through functional scenarios; idle scalability and failover are tested separately.

A stronger comparative evaluation should use repeated randomized runs and confidence intervals, a Release build, an open-loop generator in addition to `wrk`, multiple payload sizes and backend delays, and a direct HAProxy experiment with the same backend service and workload. NGINX was omitted from this final run because Docker was unavailable. No direct HAProxy benchmark was performed, and no claim is made that the project outperforms either product in general.

VII. Limitations and Conclusion

This paper compared the final project against two studies along their actual points of overlap: routing-policy/workload sensitivity and connection-processing architecture. The project demonstrates a functional C++17 TCP load balancer with three routing policies, active TCP health checks, initial-connect passive retry, two proxy architectures, multi-worker epoll, and half-close handling. Its final run favors epoll over the threaded mode in observed throughput and P99 latency at 800 connections, while the idle sweep shows a clear process-thread and memory advantage for epoll in this host test.

The evidence remains limited by a non-optimized build, one three-second throughput sample per cell, local synthetic responses, and the absence of a direct HAProxy comparison. The research papers are contextual comparisons, not numerical baselines. The next step is a controlled workload study that combines the routing-policy questions raised by Rawls and Salehi with the architecture comparison motivated by Pariag *et al.*, while reporting repeated measurements and uncertainty.

References

[1] C. Rawls and M. A. Salehi, "Load Balancer Tuning: Comparative Analysis of HAProxy Load Balancing Methods," arXiv:2212.14198, Dec. 2022. [Online]. Available: https://arxiv.org/abs/2212.14198

[2] D. Pariag, T. Brecht, A. S. Harji, P. A. Buhr, and A. Shukla, "Comparing the performance of web server architectures," in *Proc. EuroSys*, 2007, pp. 231–243, doi: 10.1145/1272998.1273021.